UNIT 2.Control Statements in Python

Introduction

Control statements in Python are used to control the flow of execution of a program. They determine which statements should be executed and how many times. These statements make programs flexible and capable of decision-making.

Types of Control Statements

Python control statements are mainly divided into three types:

1. **Conditional (Decision-Making) Statements**
2. **Looping (Iterative) Statements**
3. **Jump Statements**

---

1. Conditional Statements

These statements execute a block of code based on a condition.

a) if Statement

Executes a block only if the condition is true.

```
x = 10
if x > 5:
    print("x is greater than 5")
```

---

b) if…else Statement

Executes one block if condition is true, otherwise another block.

```
x = 3
if x % 2 == 0:
    print("Even")
else:
    print("Odd")
```

---

c) if…elif…else Statement

Used to check multiple conditions.

```
x = 0
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")
```

---

d) Nested if Statement

An if statement inside another if.

```
x = 10
if x > 0:
    if x % 2 == 0:
        print("Positive Even")
```

---

2. Looping Statements

Loops are used to execute a block of code repeatedly.

a) while Loop

Repeats execution as long as condition is true.

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

---

b) for Loop

Used to iterate over a sequence like list, string, or range.

```
for i in range(1, 6):
    print(i)
```

---

3. Jump Statements

These statements alter the normal flow of execution.

a) break

Terminates the loop.

```
for i in range(5):
    if i == 3:
```

```
        break
    print(i)
```

---

b) continue

Skips current iteration.

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

---

c) pass

Does nothing (placeholder statement).

```
for i in range(5):
    pass
```

Advantages of Control Statements

- Helps in decision making
- Reduces code repetition
- Improves readability
- Enables complex logic implementation

✅ ARRAYS AND STRINGS IN PYTHON

◆ 2 Mark Answers

1. What is an Array (List) in Python?

An array (called a *list* in Python) is a collection of elements stored in a single variable.

---

2. What is a String?

A string is a sequence of characters enclosed in single, double, or triple quotes.

Arrays (Lists) in Python

Definition

A list is an ordered, mutable collection of elements.

Example

```
arr = [10, 20, 30, 40]
```

Operations

- Access → `arr[0]`
- Add → `arr.append(50)`
- Remove → `arr.remove(20)`
- Length → `len(arr)`

Strings in Python

Example

```
s = "Python"
```

Basic Operations

- Access → `s[0]`
- Length → `len(s)`
- Concatenation → `s1 + s2`
- Repetition → `"Hi" * 3`

String Methods

```
"hello".upper()
"HELLO".lower()
"python".capitalize()
```

Arrays and Strings in Python

Introduction

Arrays (lists) and strings are important data structures in Python used to store and manipulate data efficiently.

---

1. Arrays (Lists)

Definition

A list is a collection of elements stored in a single variable.

Syntax

```
arr = [1, 2, 3, 4]
```

Operations

- Access → `arr[index]`
- Insert → `arr.insert(1, 100)`
- Append → `arr.append(50)`
- Delete → `arr.remove(2)`
- Length → `len(arr)`

Advantages

- Dynamic size
- Easy to modify
- Stores multiple values

---

2. Strings

Definition

A string is a sequence of characters.

Creating Strings

```
s1 = "Hello"
s2 = 'World'
```

Accessing Elements

```
print(s1[0])   # H
print(s1[-1])  # o
```

Traversing a String

```
for ch in s1:
    print(ch)
```

String Operations

1. Concatenation → `"Hello" + "World"`
2. Repetition → `"Hi" * 3`
3. Membership → `"H" in "Hello"`
4. Slicing → `s[1

Arrays and strings in python in python as 15 mark

Here is a **complete 15-mark answer** on *Arrays and Strings in Python* written in exam format 👇

✅ Arrays and Strings in Python

Introduction

In Python, **arrays and strings** are important data structures used to store and manipulate data.
Arrays (commonly implemented using lists) store multiple values, while strings store sequences of characters.

◆ 1. Arrays in Python

Definition

An array is a collection of elements stored in a single variable.
In Python, arrays are generally implemented using **lists**.

Creating an Array (List)

```
arr = [10, 20, 30, 40]
```

Accessing Elements

```
print(arr[0])   # 10
print(arr[-1])  # 40
```

Array Operations

1. Adding Elements

```
arr.append(50)
```

2. Inserting Elements

```
arr.insert(1, 15)
```

3. Removing Elements

```
arr.remove(20)
```

4. Length of Array

```
len(arr)
```

5. Traversing an Array

```
for i in arr:
    print(i)
```

Advantages of Arrays

- Store multiple values in one variable
- Easy to access using index
- Dynamic and flexible

◆ 2. Strings in Python

Definition

A string is a sequence of characters enclosed in quotes.

Creating Strings

```
s1 = "Hello"
s2 = 'Python'
```

Accessing Characters

```
print(s1[0])   # H
print(s1[-1])  # o
```

String Operations

1. Concatenation

```
print("Hello" + "World")
```

2. Repetition

```
print("Hi" * 3)
```

3. Membership

```
print("H" in "Hello")
```

4. Slicing

```
s = "Python"
print(s[1:4])   # yth
```

Traversing a String

```
for ch in "Python":
    print(ch)
```

String Methods

```
s = "python"

print(s.upper())      # PYTHON
print(s.lower())      # python
print(s.capitalize()) # Python
print(s.replace("p", "P"))
```

String Functions vs Methods

- **Function** → works independently

```
len("Hello")
```

- **Method** → works with object

```
"hello".upper()
```

◆ Differences Between Arrays and Strings

Arrays (Lists)	Strings
Store multiple data types	Store only characters
Mutable (can change)	Immutable (cannot change)
Use []	Use quotes

◆ Advantages

- Efficient data storage
- Easy manipulation
- Supports indexing and slicing

✅ Range Function in Python (15 Marks Answer)

Introduction

The `range()` function in Python is used to generate a sequence of numbers. It is commonly used with loops, especially the **for loop**, to control the number of iterations.

Definition

The `range()` function returns a sequence of numbers starting from a given value up to (but not including) a specified limit.

Syntax

```
range(start, stop, step)
```

- **start** → Starting value (default is 0)
- **stop** → Ending value (not included)
- **step** → Increment/decrement value (default is 1)

Types of range() Usage

1. range(stop)

Generates numbers from 0 to stop-1.

```
for i in range(5):
    print(i)
```

Output: 0 1 2 3 4

---

2. range(start, stop)

Generates numbers from start to stop-1.

```
for i in range(1, 6):
    print(i)
```

Output: 1 2 3 4 5

---

3. range(start, stop, step)

Generates numbers with step value.

```
for i in range(1, 10, 2):
    print(i)
```

Output: 1 3 5 7 9

Negative Step (Reverse Order)

```
for i in range(10, 0, -1):
    print(i)
```

Output: 10 9 8 7 6 5 4 3 2 1

Converting range to List

```
list(range(5))
```

Output: [0, 1, 2, 3, 4]

Uses of range() Function

- Used in loops for iteration
- Generates number sequences
- Helps in indexing and counting
- Used in programs like summation, tables, patterns

Advantages

- Saves memory (does not store all values at once)
- Easy to use
- Flexible with start, stop, and step

✅ Creating, Initializing and Accessing Elements of a String in Python (15 Marks)

Introduction

A string in Python is a sequence of characters used to store and manipulate text. Strings are one of the most commonly used data types in Python and are **immutable**, meaning their values cannot be changed after creation.

◆ 1. Creating and Initializing Strings

Strings can be created using single quotes, double quotes, or triple quotes.

a) Using Single Quotes

```
s1 = 'Hello'
```

b) Using Double Quotes

```
s2 = "Python"
```

c) Using Triple Quotes

Used for multi-line strings.

```
s3 = '''This is
a multi-line
string'''
```

---

d) Empty String Initialization

```
s = ""
```

---

e) String with Special Characters

```
s = "Hello\nWorld"
```

◆ 2. Accessing Elements of a String

Strings are indexed, meaning each character has a position.

---

a) Indexing (Positive Index)

```
s = "Python"
print(s[0])   # P
print(s[1])   # y
```

---

b) Negative Indexing

```
print(s[-1])  # n
print(s[-2])  # o
```

---

c) Slicing Strings

Used to access a range of characters.

```
print(s[0:3])   # Pyt
print(s[2:5])   # tho
```

---

d) Step in Slicing

```
print(s[0:6:2])   # Pto
```

◆ 3. Traversing a String

Traversing means accessing each character one by one.

Using for loop

```
for ch in "Python":
    print(ch)
```

Using while loop

```
i = 0
s = "Python"
while i < len(s):
    print(s[i])
    i += 1
```

◆ 4. Important Points

- Strings are **immutable**
- Indexing starts from **0**
- Supports **positive and negative indexing**
- Slicing does not include the last index

◆ 5. Advantages of Strings

- Easy to use
- Supports many built-in functions
- Useful for text processing